

PHỤ LỤC C. CÁC CẤU TRÚC DỮ LIỆU ĐẶC BIỆT

Trong phụ lục này, chúng ta sẽ xem xét rất nhanh các cấu trúc dữ liệu được TensorFlow hỗ trợ, ngoài các tensor số thực hoặc số nguyên thông thường. Điều này bao gồm chuỗi, tensor rỗng, tensor thưa, mảng tensor, tập hợp và hàng đợi.

Chuỗi (Strings)

Các tensor có thể chứa chuỗi byte, điều này đặc biệt hữu ích cho xử lý ngôn ngữ tự nhiên (xem Chương 16):

```
import tensorflow as tf
>>> tf.constant(b"hello world")
<tf.Tensor: shape=(), dtype=string, numpy=b'hello world'>
```

Nếu bạn cố gắng xây dựng một tensor với một chuỗi Unicode, TensorFlow sẽ tự động mã hóa nó sang UTF-8:

```
>>> tf.constant("café")
<tf.Tensor: shape=(), dtype=string, numpy=b'caf\xc3\xa9'>
```

Cũng có thể tạo các tensor biểu diễn chuỗi Unicode. Chỉ cần tạo một mảng các số nguyên 32-bit, mỗi số biểu diễn một điểm mã Unicode:

```
>>> u = tf.constant([ord(c) for c in "café"])

>>> u
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([ 99,  97, 102, 233], dtype=int32)>
```

Gói `tf.strings` chứa một số hàm để thao tác với các tensor chuỗi, chẳng hạn như `length()` để đếm số byte trong một chuỗi byte (hoặc số điểm mã nếu bạn đặt `unit="UTF8_CHAR"`), `unicode_encode()` để chuyển đổi một tensor chuỗi Unicode (tức là tensor `int32`) thành một tensor chuỗi byte, và `unicode_decode()` để làm ngược lại:

```
>>> b = tf.strings.unicode_encode(u, "UTF-8")

>>> b
<tf.Tensor: shape=(), dtype=string, numpy=b'caf\xc3\xa9'>

>>> tf.strings.length(b, unit="UTF8_CHAR")
<tf.Tensor: shape=(), dtype=int32, numpy=4>

>>> tf.strings.unicode_decode(b, "UTF-8")
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([ 99,  97, 102, 233], dtype=int32)>
```

Bạn cũng có thể thao tác với các tensor chứa nhiều chuỗi:

```
>>> p = tf.constant(["Café", "Coffee", "caffè", "咖啡"])

>>> tf.strings.length(p, unit="UTF8_CHAR")
<tf.Tensor: shape=(4,), dtype=int32, numpy=array([4, 6, 5, 2], dtype=int32)>

>>> r = tf.strings.unicode_decode(p, "UTF8")

>>> r
<tf.RaggedTensor [[67, 97, 102, 233], [67, 111, 102, 102, 101, 101], [99, 97, 102, 102, 232], [21654, 21857]]>
```

Lưu ý rằng các chuỗi đã giải mã được lưu trữ trong một RaggedTensor. RaggedTensor là gì?

Tensor Răng Cưa (Ragged Tensors)

Một tensor răng cưa là một loại tensor đặc biệt biểu diễn một danh sách các mảng có kích thước khác nhau. Tổng quát hơn, nó là một tensor có một hoặc nhiều chiều răng cưa, nghĩa là các chiều mà các lát cắt của chúng có thể có độ dài khác nhau. Trong tensor răng cưa *r*, chiều thứ hai là một chiều răng cưa. Trong tất cả các tensor răng cưa, chiều đầu tiên luôn là một chiều thông thường (còn gọi là chiều đồng nhất).

Tất cả các phần tử của tensor răng cưa *r* đều là các tensor thông thường. Ví dụ, hãy xem phần tử thứ hai của tensor răng cưa:

```
>>> r[1]
<tf.Tensor: shape=(6,), dtype=int32, numpy=array([ 67, 111, 102, 102, 101, 101], dtype=int32)>
```

Gói `tf.ragged` chứa một số hàm để tạo và thao tác với các tensor răng cưa. Hãy tạo một tensor răng cưa thứ hai bằng cách sử dụng `tf.ragged.constant()` và nối nó với tensor răng cưa đầu tiên, dọc theo trục 0:

```
>>> r2 = tf.ragged.constant([[65, 66], [], [67]])

>>> tf.concat([r, r2], axis=0)
<tf.RaggedTensor [[67, 97, 102, 233], [67, 111, 102, 102, 101, 101], [99, 97, 102, 102, 232], [21654, 21857], [65, 66], [], [67]]>
```

Kết quả không quá ngạc nhiên: các tensor trong *r2* được nối vào sau các tensor trong *r* dọc theo trục 0. Nhưng điều gì sẽ xảy ra nếu chúng ta nối *r* và một tensor răng cưa khác dọc theo trục 1?

```
>>> r3 = tf.ragged.constant([[68, 69, 70], [71], [], [72, 73]])

>>> print(tf.concat([r, r3], axis=1))
<tf.RaggedTensor [[67, 97, 102, 233, 68, 69, 70], [67, 111, 102, 102, 101, 101, 101, 71], [99, 97, 102, 102, 232], [21654, 21857, 72, 73]]>
```

Lần này, lưu ý rằng tensor thứ i trong r và tensor thứ i trong $r3$ đã được nối với nhau. Điều này bất thường hơn, vì tất cả các tensor này có thể có độ dài khác nhau.

Nếu bạn gọi phương thức `to_tensor()`, tensor răng cưa sẽ được chuyển đổi thành một tensor thông thường, đệm các tensor ngắn hơn bằng số 0 để có được các tensor có độ dài bằng nhau (bạn có thể thay đổi giá trị mặc định bằng cách đặt đối số `default_value`):

```
>>> r.to_tensor()
<tf.Tensor: shape=(4, 6), dtype=int32, numpy=
array([[ 67,   97,  102,  233,    0,    0],
       [ 67,  111,  102,  102,  101,  101],
       [ 99,   97,  102,  102,  232,    0],
       [21654, 21857,    0,    0,    0,    0]], dtype=int32)>
```

Nhiều phép toán TF hỗ trợ các tensor răng cưa. Để biết danh sách đầy đủ, xem tài liệu của lớp `tf.RaggedTensor`.

Tensor Thừa (Sparse Tensors)

TensorFlow cũng có thể biểu diễn hiệu quả các tensor thừa (tức là các tensor chứa chủ yếu là số không). Chỉ cần tạo một `tf.SparseTensor`, chỉ định các chỉ số và giá trị của các phần tử khác không và hình dạng của tensor. Các chỉ số phải được liệt kê theo “thứ tự đọc” (từ trái sang phải, và từ trên xuống dưới). Nếu bạn không chắc chắn, chỉ cần sử dụng `tf.sparse.reorder()`.

Bạn có thể chuyển đổi một tensor thừa thành một tensor dày đặc (tức là một tensor thông thường) bằng cách sử dụng `tf.sparse.to_dense()`:

```
>>> s = tf.SparseTensor(indices=[[0, 1], [1, 0], [2, 3]],
...                     values=[1., 2., 3.],
...                     dense_shape=[3, 4])
...
>>> tf.sparse.to_dense(s)
<tf.Tensor: shape=(3, 4), dtype=float32, numpy=
array([[0., 1., 0., 0.],
       [2., 0., 0., 0.],
       [0., 0., 0., 3.]], dtype=float32)>
```

Lưu ý rằng các tensor thừa không hỗ trợ nhiều phép toán như các tensor dày đặc. Ví dụ, bạn có thể nhân một tensor thừa với bất kỳ giá trị vô hướng nào, và bạn sẽ nhận được một tensor thừa mới, nhưng bạn không thể cộng một giá trị vô hướng vào một tensor thừa, vì điều này sẽ không trả về một tensor thừa:

```
>>> s * 42.0
<tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x...>

>>> s + 42.0
[...]
TypeError: unsupported operand type(s) for +: 'SparseTensor' and 'float'
```

Mảng Tensor (Tensor Arrays)

Một `tf.TensorArray` biểu diễn một danh sách các tensor. Điều này có thể tiện dụng trong các mô hình động chứa vòng lặp, để tích lũy kết quả và sau đó tính toán một số thống kê. Bạn có thể đọc hoặc ghi các tensor tại bất kỳ vị trí nào trong mảng:

```
array = tf.TensorArray(dtype=tf.float32, size=3)
array = array.write(0, tf.constant([1., 2.]))
array = array.write(1, tf.constant([3., 10.]))
array = array.write(2, tf.constant([5., 7.]))
tensor1 = array.read(1) #=> trả về (và xóa thành 0!) tf.constant([3., 10.])
print(tensor1)
```

Theo mặc định, việc đọc một mục cũng sẽ thay thế nó bằng một tensor có cùng hình dạng nhưng chứa đầy số không. Bạn có thể đặt `clear_after_read` thành `False` nếu bạn không muốn điều này.

Theo mặc định, một `TensorArray` có kích thước cố định được đặt khi tạo. Thay vào đó, bạn có thể đặt `size=0` và `dynamic_size=True` để cho phép mảng tự động mở rộng khi cần. Tuy nhiên, điều này sẽ cản trở hiệu suất, vì vậy nếu bạn biết kích thước trước, tốt hơn là sử dụng một mảng có kích thước cố định. Bạn cũng phải chỉ định `dtype`, và tất cả các phần tử phải có cùng hình dạng với phần tử đầu tiên được ghi vào mảng.

Bạn có thể xếp chồng tất cả các mục thành một tensor thông thường bằng cách gọi phương thức `stack()`:

```
>>> array.stack()
<tf.Tensor: shape=(3, 2), dtype=float32, numpy=
array([[1., 2.],
       [0., 0.],
       [5., 7.]], dtype=float32)>
```

Tập hợp (Sets)

TensorFlow hỗ trợ các tập hợp số nguyên hoặc chuỗi (nhưng không phải số float). Nó biểu diễn các tập hợp bằng cách sử dụng các tensor thông thường. Ví dụ, tập hợp {1, 5, 9} chỉ được biểu diễn dưới dạng tensor `[[1, 5, 9]]`. Lưu ý rằng tensor phải có ít nhất hai chiều, và các tập hợp phải nằm ở chiều cuối cùng. Ví dụ, `[[1, 5, 9], [2, 5, 11]]` là một tensor chứa hai tập hợp độc lập: {1, 5, 9} và {2, 5, 11}.

Gói `tf.sets` chứa một số hàm để thao tác với các tập hợp. Ví dụ, hãy tạo hai tập hợp và tính hợp của chúng (kết quả là một tensor thưa, vì vậy chúng ta gọi `to_dense()` để hiển thị nó):

```
>>> a = tf.constant([[1, 5, 9]])
>>> b = tf.constant([[5, 6, 9, 11]])
>>> u = tf.sets.union(a, b)
```

```
>>> u
<tensorflow.python.framework.sparse_tensor.SparseTensor object at 0x...>

>>> tf.sparse.to_dense(u)
<tf.Tensor: shape=(1, 5), dtype=int32, numpy=array([[ 1,  5,  6,  9, 11]], dtype=int32)>
```

Bạn cũng có thể tính hợp của nhiều cặp tập hợp đồng thời. Nếu một số tập hợp ngắn hơn các tập hợp khác, bạn phải đệm chúng bằng một giá trị đệm, chẳng hạn như 0:

```
>>> a = tf.constant([[1, 5, 9], [10, 0, 0]])

>>> b = tf.constant([[5, 6, 9, 11], [13, 0, 0, 0]])

>>> u = tf.sets.union(a, b)

>>> tf.sparse.to_dense(u)
<tf.Tensor: shape=(2, 5), dtype=int32, numpy=
array([[ 1,  5,  6,  9, 11],
       [ 0, 10, 13,  0,  0]], dtype=int32)>
```

Nếu bạn muốn sử dụng một giá trị đệm khác, chẳng hạn như -1, thì bạn phải đặt `default_value=-1` (hoặc giá trị ưa thích của bạn) khi gọi `to_dense()`.

Các hàm khác có sẵn trong `tf.sets` bao gồm `difference()`, `intersection()`, và `size()`, những hàm này tự giải thích. Nếu bạn muốn kiểm tra xem một tập hợp có chứa một số giá trị đã cho hay không, bạn có thể tính giao của tập hợp đó và các giá trị. Nếu bạn muốn thêm một số giá trị vào một tập hợp, bạn có thể tính hợp của tập hợp và các giá trị.

Hàng đợi (Queues)

Một hàng đợi là một cấu trúc dữ liệu mà bạn có thể đẩy các bản ghi dữ liệu vào, và sau đó kéo chúng ra. TensorFlow triển khai một số loại hàng đợi trong gói `tf.queue`. Chúng từng rất quan trọng khi triển khai các pipeline tải và tiền xử lý dữ liệu hiệu quả, nhưng API `tf.data` về cơ bản đã làm cho chúng trở nên vô dụng (trừ một số trường hợp hiếm hoi) vì nó đơn giản hơn nhiều để sử dụng và cung cấp tất cả các công cụ bạn cần để xây dựng các pipeline hiệu quả. Tuy nhiên, vì mục đích hoàn chỉnh, chúng ta hãy xem xét nhanh chúng.

Loại hàng đợi đơn giản nhất là hàng đợi vào trước, ra trước (FIFO). Để xây dựng nó, bạn cần chỉ định số lượng bản ghi tối đa mà nó có thể chứa. Hơn nữa, mỗi bản ghi là một tuple các tensor, vì vậy bạn phải chỉ định kiểu của mỗi tensor, và tùy chọn hình dạng của chúng. Ví dụ, đoạn mã sau đây tạo một hàng đợi FIFO với tối đa ba bản ghi, mỗi bản ghi chứa một tuple với một số nguyên 32-bit và một chuỗi. Sau đó, nó đẩy hai bản ghi vào, xem kích thước (là 2 tại thời điểm này), và kéo một bản ghi ra:

```
>>> q = tf.queue.FIFOQueue(3, [tf.int32, tf.string], shapes=[(), ()])

>>> q.enqueue([10, b"windy"])
```

```
>>> q.enqueue([15, b"sunny"])

>>> q.size()
<tf.Tensor: shape=(), dtype=int32, numpy=2>

>>> q.dequeue()
[<tf.Tensor: shape=(), dtype=int32, numpy=10>,
 <tf.Tensor: shape=(), dtype=string, numpy=b'windy'>]
```

Cũng có thể đẩy và kéo nhiều bản ghi cùng lúc bằng cách sử dụng `enqueue_many()` và `dequeue_many()` (để sử dụng `dequeue_many()`, bạn phải chỉ định đối số `shapes` khi bạn tạo hàng đợi, như chúng ta đã làm trước đó):

```
>>> q.enqueue_many([[13, 16], [b'cloudy', b'rainy']])

>>> q.dequeue_many(3)
[<tf.Tensor: shape=(3,), dtype=int32, numpy=array([15, 13, 16], dtype=int32)>,
 <tf.Tensor: shape=(3,), dtype=string, numpy=array([b'sunny', b'cloudy', b'rainy'], dtype=object)>]
```

Các loại hàng đợi khác bao gồm:

- **PaddingFIFOQueue:** Giống như `FIFOQueue`, nhưng phương thức `dequeue_many()` của nó hỗ trợ kéo ra nhiều bản ghi có hình dạng khác nhau. Nó tự động đệm các bản ghi ngắn nhất để đảm bảo tất cả các bản ghi trong lô có cùng hình dạng.
- **PriorityQueue:** Một hàng đợi kéo các bản ghi theo thứ tự ưu tiên. Ưu tiên phải là một số nguyên 64-bit được bao gồm làm phần tử đầu tiên của mỗi bản ghi. Đáng ngạc nhiên, các bản ghi có ưu tiên thấp hơn sẽ được kéo ra trước. Các bản ghi có cùng ưu tiên sẽ được kéo ra theo thứ tự FIFO.
- **RandomShuffleQueue:** Một hàng đợi mà các bản ghi của nó được kéo ra theo thứ tự ngẫu nhiên. Điều này hữu ích để triển khai một bộ đệm xáo trộn trước khi `tf.data` tồn tại.

Nếu một hàng đợi đã đầy và bạn cố gắng đẩy một bản ghi khác vào, phương thức `enqueue*()` sẽ bị đóng băng cho đến khi một bản ghi được kéo ra bởi một luồng khác. Tương tự, nếu một hàng đợi trống và bạn cố gắng kéo một bản ghi ra, phương thức `dequeue*()` sẽ bị đóng băng cho đến khi các bản ghi được đẩy vào hàng đợi bởi một luồng khác.

Phán đoán và Công thức Toán:

Phụ lục này chủ yếu mô tả các cấu trúc dữ liệu đặc biệt trong TensorFlow và cách chúng được sử dụng, không đi sâu vào các công thức toán học. Tuy nhiên, có một số điểm có thể liên hệ với khái niệm toán học và cách chúng được biểu diễn hoặc thao tác:

- **Unicode Code Points:**

- `tf.constant([ord(c) for c in "café"])`: Hàm `ord(c)` trả về giá trị số nguyên của điểm mã Unicode của ký tự `c`. Đây là một ánh xạ từ ký tự sang một số nguyên duy nhất, về bản chất là một phép biểu diễn số. Ví dụ:
 - `ord('c') = 99`
 - `ord('a') = 97`
 - `ord('f') = 102`
 - `ord('é') = 233`
- Biểu diễn UTF-8: `b'caf\xc3\xa9'` cho “café”. Đây là một cách mã hóa chuỗi Unicode thành chuỗi byte, trong đó một số ký tự (như é) yêu cầu nhiều byte.

- **Ragged Tensors:**

- Đây là cấu trúc dữ liệu để biểu diễn các mảng có độ dài không đồng nhất. Mặc dù không có công thức toán học, khái niệm này liên quan đến việc quản lý bộ nhớ và truy cập hiệu quả các dữ liệu có kích thước thay đổi.
- Khi chuyển đổi sang tensor thông thường bằng `to_tensor()`, việc đệm bằng số 0 (hoặc giá trị mặc định) có thể được coi là việc thêm các phần tử giả vào các hàng/cột ngắn hơn để chúng có cùng kích thước, làm cho tensor có hình dạng hình chữ nhật hoặc khối lập phương.

- **Sparse Tensors:**

- Biểu diễn hiệu quả các tensor chứa chủ yếu là số không. Điều này là một kỹ thuật tối ưu hóa bộ nhớ cho các ma trận thưa (sparse matrices).
- `tf.SparseTensor(indices=[[0, 1], [1, 0], [2, 3]], values=[1., 2., 3.], dense_shape=[3, 4])`: Cấu trúc này lưu trữ chỉ các phần tử khác không cùng với chỉ số của chúng và hình dạng tổng thể của tensor. Ví dụ, phần tử 1. nằm ở (0, 1), phần tử 2. ở (1, 0), v.v. Các vị trí không được liệt kê ngụ ý giá trị 0.
- Phép nhân vô hướng `s * 42.0`: Chỉ các giá trị khác không cần được nhân với vô hướng.
 - Đối với mỗi (idx, val) trong `SparseTensor`, phép toán trở thành $(idx, val \times scalar)$.
- Phép cộng vô hướng `s + 42.0` không được hỗ trợ vì nó sẽ biến hầu hết các phần tử 0 thành 42, làm cho tensor không còn thưa nữa và yêu cầu chuyển đổi sang tensor dày đặc.

- **Tập hợp (Sets):**

- Các phép toán tập hợp như hợp (union), giao (intersection), hiệu (difference), và kích thước (size) là các phép toán cơ bản trong lý thuyết tập hợp.

- `tf.sets.union(a, b)`: Tính hợp của hai tập hợp. Nếu A và B là hai tập hợp, hợp của chúng là $A \cup B = \{x \mid x \in A \text{ hoặc } x \in B\}$.
- **Hàng đợi (Queues):**
 - Các hàng đợi (FIFO, Priority, RandomShuffle) là các cấu trúc dữ liệu trừu tượng với các quy tắc cụ thể cho việc thêm (enqueue) và xóa (dequeue) phần tử.
 - **FIFO (First-In, First-Out)**: Phần tử được thêm vào đầu tiên sẽ được xóa ra đầu tiên. Giống như một hàng người.
 - **PriorityQueue**: Các phần tử được xóa ra dựa trên một giá trị ưu tiên. Mặc dù văn bản nói rằng “lower priority will be dequeued first” (ưu tiên thấp hơn sẽ được kéo ra trước), điều này có thể hiểu là “giá trị số ưu tiên thấp hơn” tương ứng với “mức độ ưu tiên cao hơn” trong một số ngữ cảnh.
 - **RandomShuffleQueue**: Các phần tử được xóa ra theo thứ tự ngẫu nhiên, ngụ ý một quá trình lấy mẫu ngẫu nhiên từ tập hợp các phần tử trong hàng đợi.

Tóm lại, phần phụ lục này trình bày các cấu trúc dữ liệu được thiết kế để xử lý hiệu quả các loại dữ liệu cụ thể (chuỗi, dữ liệu có kích thước không đồng nhất, dữ liệu thưa, dữ liệu động) trong bối cảnh TensorFlow, phản ánh sự giao thoa giữa khoa học máy tính và các nguyên tắc toán học cơ bản.